# Comparison of Viewshed Algorithms on Regular Spaced Points

Branko Kaučič[1]
Borut Žalik[2]

[1]Department of Mathematics, Faculty of Education
[2]Faculty of Electrical Engineering and Computer Science
University of Maribor, Slovenia

## Abstract

Viewshed is a basic visibility structure and represents all visible points from a given viewpoint. The paper gives a survey of non-parallel viewshed algorithms using regular spaced points. At first, a comprehensive background of the problem is given, then the existing methods (*R3*, *R2*, *XDraw*, method based on the line rasterisation) are introduced briefly. They have been implemented in 11 variants. Real terrain data and fractal generated terrains have been used for their comparison against spent CPU time and the accuracy of a computed visibility information.

**Keywords:** Computational Geometry, Viewshed, Visibility, Regular Spaced Points, Survey

## 1 Introduction

Visibility analysis is an elementary part of many applications of computational geometry and computer graphics, as for example: special rendering methods, computer games, navigational and visibility-based engineering applications. In the paper, the cases where viewpoint lies inside the object's domain (for example on or above a terrain) are considered. As such, the problem differs from the well-known hidden-surface removal algorithms.

Visibility problems are roughly divided into two groups: computation of visibility information from a given viewpoint and the use of the computed visibility information. The most frequently computed visibility information are viewsheds, horizons, and visibility indices. The computed visibility information is used to solve different (possibly optimisation) problems. The examples are determining the minimum number of observation facilities (radar, TV, telephone transmitters, etc.), or computing the optimum path with specific visibility characteristics (scenic paths, hidden paths, etc.). For a survey on this subject see [2, 3].

In the paper, a survey of viewshed algorithms is given. The viewshed is a basic visibility structure needed at determining visibility indices, and at most optimisation problems. It represents all visible points from a given viewpoint. Since computing the viewshed is naturally related with terrains, digital terrains represented by regular spaced points are used in the continuation.

The paper is organised in five sections. Necessarily theoretical background is introduced in Section 2. In Section 3, all known (by authors knowledge) viewshed algorithms are explained briefly. In Section 4 the algorithms are compared using real terrain data and fractal generated terrains. In the last, the fifth Section, the paper is summarised.

## 2 Preliminaries

Terrain data are related with a 3D configuration of the Earth's surface. The geometry of a terrain is usually modeled as a 2,5D surface, i.e. a surface in 3D space described by a surjective bivariate function. A model representing a terrain relief using a finite number of discrete samples is known as a Digital Terrain Model (DTM). Two classes of DTMs are widely used [2]: triangulated irregular networks (TIN) and regular square grids (RSG). Within recent trends in Geographical Information Systems RSGs are prefered, and therefore they are used in the paper (Figure 1).

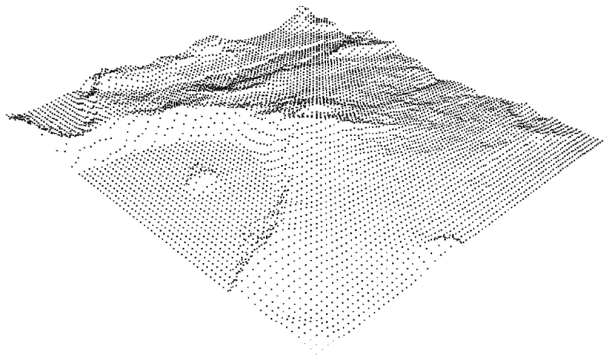


Figure 1: An example of a terrain represented by regular spaced points

DTM is defined by a set of points $p \equiv (x_p, y_p, z_p)$. Two points $p_1$ and $p_2$ are mutually visible (intervisible) iff ev-

ery point $q \equiv (x_q, y_q, z_q) = p_1 + t(p_2 - p_1), 0 < t < 1$, lies above the corresponding point $p_q$ of the terrain, i.e. $z > z_q$ [2]. In other words, two points are mutually visible when straight line $p_1p_2$ lies above the terrain and touches it only at its end points. If one of these two points is a viewpoint (labeled $O$ in the continuation), the ray from $O$ through the second point is called a line-of-sight (LOS). If LOS $Op_2$ is not obstructed by the terrain (or by the objects on the terrain), point $p_2$ is visible.

The definition of the visibility can be further extended to the points called targets. The targets are not parts of the terrain and lie above it. An example of the visibility of terrain points and targets can be seen in Figure 2. We have

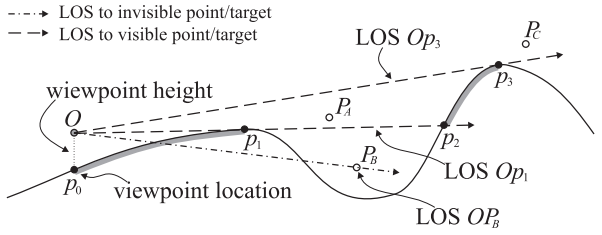



Figure 2: Visibility of terrain points and targets

a terrain, viewpoint $O$ and three targets: $P_A$, $P_B$, and $P_C$. From the viewpoint, all terrain points shaded with thick gray lines are visible. From point $p_1$ to point $p_2$ all terrain points below LOS $Op_1$ are invisible. Similarly, target $P_B$ is not visible because LOS through $P_B$ does not lie above the terrain. As seen, point $p_1$ in Figure 2 represents the highest obstacle at determining visibility of the points on its right side.

Let us consider one step further. If we know how to compute visibility problems in 1,5D (Figure 2), how do we compute the visibility in 2,5D? Figure 3 considers that problem. Let $E$ be the plane, perpendicular to plane $xy$,

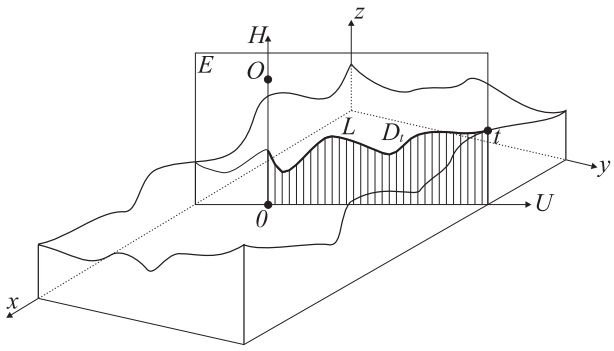
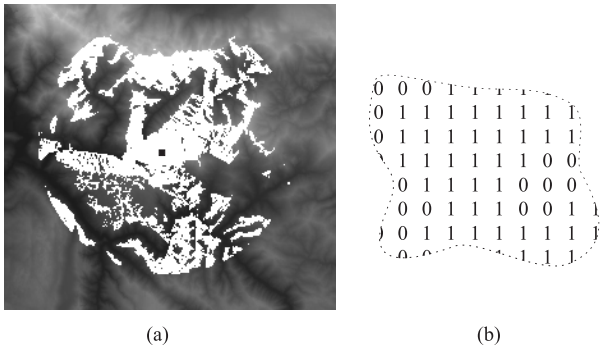



Figure 3: The visibility problem in 2,5D, translated to the visibility problem in 1,5D

passing through viewpoint $O$ and point $t$. Plane $E$ consequently defines a 1,5D coordinate system $HU$. Its origin coincides with the viewpoint's vertical projection on the plane $xy$. The intersection between $E$ and the surface of a terrain defines curve $L$ which visibility is investigated. By rotating plane $E$ around axis $H$ for a small discrete steps, a set of discrete curves approximating terrain of interest is obtained. In this way, the 2,5D visibility problem is transformed to a set of 1,5D visibility problems, i.e. for each curve $L$. Curve $L$ is represented by a discrete profile $D_t$ (see Figure 3).

In general, the visibility of terrain, or objects on it (i.e. houses, transmitters, etc.), is solved online and stored in visibility structures. The basic visibility structure is a viewshed. It represents the region of a terrain, visible from a given viewpoint. An example of the viewshed on RSG can be seen in Figure 4 together with its usual representation as a matrix of boolean values. Figure 4 (a) simulates a view with a radius 10 km in the north-west part of Slovenia, and Figure 4 (b) shows an excerpt of the resulting matrix ("0" represents invisible points, and "1" visible points).




(a) (b)

Figure 4: An example of viewshed: (a) a view from a location in Slovenia, (b) internal data structure

Although the viewshed in Figure 4 is computed for a circular shape, it can also have different shapes as will be shown in the continuation. Some algorithms were originally developed for a circular or a square shape, but can be easily adopted to cover arbitrary shapes.

# 3 The viewshed methods

Not many researchers coped with computing the viewshed. Their primary motivation was to efficiently reduce the following two problems:

- The quality of the object's representation depends on the resolution of sampled data. Better resolution means more accurate visibility analysis, which is desired by the users.

- On the other side, better representation means more memory and more computational time. The drawback of the later is noticeable even at the small changes of the resolution.

In the continuation, the most popular approaches are considered briefly. They are divided into exact and approximative. The exact method is only one, while all others are approximative.

a) **Exact method R3** (*R3*): The viewshed is computed in a classical way [4, 5, 8]. For every point within an area of interest, the LOS is calculated and checked whether it is obstructed. Figure 5 shows its behaviour in the first octant. With the respect to the octants, heights at corresponding $x$- and $y$-crossings (see Figure 5) are estimated. The estimation is done between the two closest points (in the same direction as shown in Figure 5 with the marked $x$-crossings) and is usually done with the linear interpolation. If the minimum, the maximum, or the average value is used instead of the interpolation, the method becomes approximative.

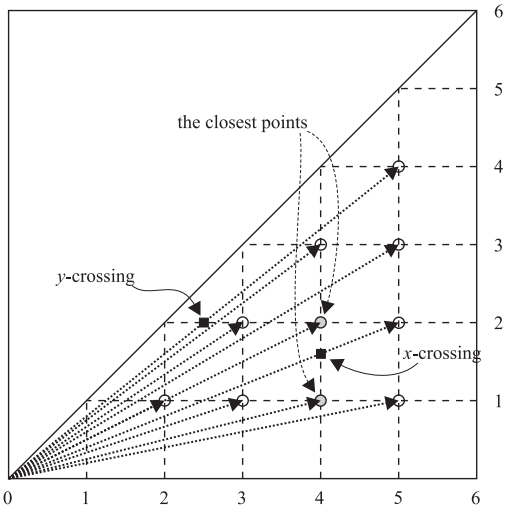



Figure 5: Method *R3* in the first octant

b) **Approximative method R2** (*R2*): At method *R3* the order in which LOSs to points are calculated is not important. Let us consider Figure 6 and the fact that the LOSs to points from $a$ to $d$ are determined before any other points in the first octant [4]. Suppose that during the computation of their LOSs, the heights in all crossings are stored (black squares in Figure 6). Then, the approximative information used to determine the visibility of points $e$, $f$, and $g$ is already provided. Namely, the visibility of every point is determined approximatively as the visibility of the closest crossing. For example, the visibility of point $e$ is computed in the crossing above point $e$ (Figure 6). The pattern is repeated for the other points, as $h$, $i$, etc. Such process implies to the more general approach of computing the LOSs only to the boundary points of an area of interest.

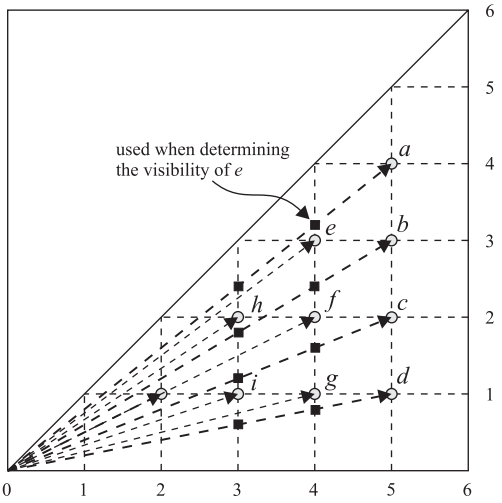



Figure 6: Method *R2* in the first octant

c) **Approximative method XDraw** (*XDraw*): The visibility and the heights of the LOSs are computed for square rings around the viewpoint, as shown in Figure 7 [4]. Each point stores an additional information of the highest LOS to it up to now, in that direction. When the visibility of some point in ring $i$ is determined, the corresponding point or the pair of points in inner ring $i-1$ is used. If the point is invisible, the LOS to that point is set from the LOS passing over inner point or pair of points. Otherwise, its LOS is stored. Single inner point is used in eight sky directions, otherwise two inner points are used (points $(2,1)$ and $(3,1)$ in Figure 7 when the visibility of point $(5,2)$ is determined). In the later, the height between the inner points can be estimated similar as in method *R3* with the linear interpolation, or using the minimum, the maximum, or the average value.

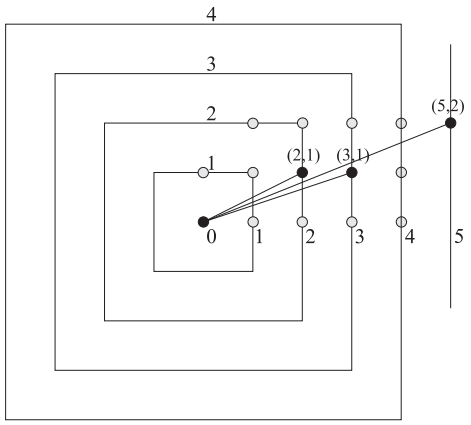



Figure 7: Visibility rings in method *XDraw*

d) **Approximative methods with the line rasterisation** (*Bresenham, 4WS, 8WS*): Discretisation of profile $D_t$ (see Section 2) can be done using the well-known line rasterisation algorithms (Figure 8) [1, 8]. Instead of dealing with

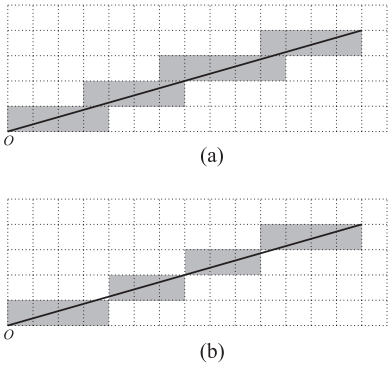



Figure 8: Different line rasterisations: (a) 4WS, (b) Bresenham, 8WS

the estimated visibility in the crossings, the visibility of points from the rasterisation of LOS's vertical projection is used. The most known approach applies the famous Bresenham algorithm. The alternatives are 4WS (4-way stepping) and 8WS (8-way stepping) algorithms with the midpoint method [9]. Figure 8 (a) shows the 4WS rasterisation and Figure 8 (b) the Bresenham and the 8WS rasterisation.

### 3.1 Few thoughts on the methods

The method *R3* gives an exact result but works in $O(r^3)$ time, where $r$ represents an average radius of view range. Its benefit is that it uses only points that are in immediate neighbourhood (within distance 1) of LOSs.

That is not the case with approximative methods *R2* and *XDraw*, where the wavefront propagation is applied. The visibility information from some point can be propagated as shown in Figure 9. Consequently, the points not being a

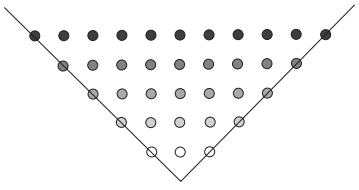


Figure 9: Wavefront propagation

part of the immediate neighbourhood influence on the visibility. In this way, terrain points are divided to proper and improper points. Such influence is especially noticeable when an improper point contains a dominant property, and it is reduced to the minimum, if the linear interpolation is used at the computation. Another important issue is that when using the minimum, the points recognised as invisible are indeed invisible, and when using the maximum, the points recognised as visible are indeed visible.

The second type of the approximation is done with the line rasterisation. Its main drawback is that rasterised lines overlap partially. Because in general (in algorithms), there is no checking whether the visibility of a terrain point has been determined, in the next iteration the visibility of the same point can be computed again, and perhaps with the opposite resulting visibility. However, the worst case that could happen is if some terrain points are left out. For example, that happens when the boundary points of the viewshed are 8-connected and 8WS lines are used.

Because all approximative algorithms operate by computing the LOSs to the boundary points, they exhibit $O(r^2)$ time complexity.

## 4 Testing the algorithms

To determine how good the methods actually are, they have been implemented in C++ and mutually compared by the authors of the paper. The selection of the corresponding algorithm for the visibility problem in 1,5D (see Section 2) has been done according to the results given by Kaučič and Žalik [6]. Authors claimed that two algorithms (the biggest slope of all LOSs and the improved incremental approach) are the most appropriate. In our tests, the biggest slope of all LOSs has been selected.

Because the best and the worst cases of 1,5D algorithms are close [6], the viewshed algorithms have been tested only on data, expected in real applications. Two terrain types have been used: fractal generated terrains based on the Brownian fractal motion [7] and the digital terrain relief model of Slovenia at resolutions $100m \times 100m$ and $25m \times 25m$, provided by the Surveying and Mapping Authority of the Republic Slovenia [10]. Data-set of Slovenia was considered as a representative because of the variety of its terrain characteristics: mountains at north-west, lowland in the east, and hills and valleys all over the rest of the country. Figure 10 shows the model of Slovenia and the part with the $25m \times 25m$ resolution.

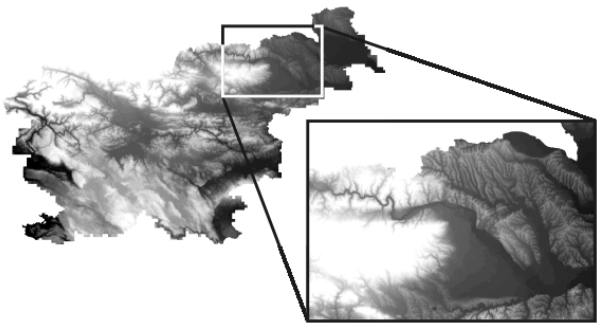


Figure 10: Digital terrain relief model of Slovenia

Although all algorithms can compute the viewshed in circular and square shape, the tests have been performed on square viewsheds with radius $r$ for 20 different values. Each test has been repeated for identical $r$ for 10 random viewpoint locations. A portable computer with Intel Celeron 500 MHz, 64 MB memory and Windows 98 has been used. In all cases, the viewpoint has been set to simulate a human view, 1.6m above the corresponding point.

All methods from Section 3 have been implemented in 11 variants. The exact algorithm *R3Int* uses the linear interpolation. In algorithms *R3Avg*, *R3Min* and *R3Max* the linear interpolation is replaced by the average, the minimum and the maximum, respectively. The approximative method *R2* has only one variant, while the algorithms *XDrawInt*, *XDrawAvg*, *XDrawMin* and *XDrawMax* represent method *XDraw* with the linear interpolation, the average, the minimum and the maximum, respectively. Among the methods based on the line rasterisation, three algorithms have been implemented: *Bresenham*, *4WS* and *8WS*.

The main goal of the tests have been to find the time ratio and the accuracy ratio of all approximative algorithms against exact algorithm *R3Int*. The error ratio has been observed as a number of wrong computations of visibility of points against all points in the viewshed. In the real terrain data, height values have been in the range from 0 m to 2864 m, and the average viewpoint height has been 457,8 m. Similarily, in the fractal generated terrains, height values have been in the range from 0 m to 1000 m, the average viewpoint has been 471,9 m, and the average $H$ [7] has been 0,65 (0.0 means smooth and 1.0 rough terrain). As expected, the same results have been obtained at the both terrain types.

Table 1 shows the excerpt of average run-times of the algorithms in seconds for five different $r$. Table 2 gives their obtained average number of visible points. Table 3 shows the average number of points that were identified as visible although they were invisible, while Table 4 shows the average number of points that were identified as invisible although they were visible according to algorithm *R3Int*. Graphical representations of these results are shown in Figures 11, 12, 13 and 14. Because tests have shown the same behaviour for *Bresenham* and *8WS*, the results for *Bresenham* are not explicitly given.

Although the spent CPU times in the tables are relative small, in real applications larger $r$ are used, and the viewsheds are computed for every terrain point in order to form visibility indices. The spent CPU time in such applications arises steeply. At other viewpoint heights the same spent CPU time of the methods is expected [6].

As expected, from the results it can be seen, that the algorithms based on the minimum and the maximum, do not produce certain type of error. For example, *R3Min* never computes point as visible if that point is actually invisible.

| Algorithm\$r$ | 50 | 100 | 150 | 200 | 250 |
|---|---|---|---|---|---|
| *R3Int* | 0,279 | 2,164 | 7,305 | 17,325 | 33,844 |
| *R3Avg* | 0,276 | 2,078 | 6,848 | 16,268 | 31,857 |
| *R3Max* | 0,24 | 2,011 | 6,822 | 16,292 | 31,87 |
| *R3Min* | 0,233 | 2,0 | 6,854 | 16,293 | 31,912 |
| *R2* | 0,022 | 0,08 | 0,198 | 0,374 | 0,606 |
| *XDrawInt* | 0,001 | 0,04 | 0,077 | 0,188 | 0,275 |
| *XDrawAvg* | 0,001 | 0,037 | 0,078 | 0,145 | 0,255 |
| *XDrawMax* | 0,01 | 0,034 | 0,069 | 0,134 | 0,24 |
| *XDrawMin* | 0,012 | 0,041 | 0,09 | 0,153 | 0,252 |
| *4WS* | 0,001 | 0,034 | 0,099 | 0,162 | 0,273 |
| *8WS* | 0,001 | 0,033 | 0,056 | 0,131 | 0,198 |

Table 1: Average spent CPU times (s) of viewshed algorithms at viewpoint height +1,6 m

| Algorithm\$r$ | 50 | 100 | 150 | 200 | 250 |
|---|---|---|---|---|---|
| *R3Int* | 1340,2 | 2979,9 | 4154,8 | 4461,7 | 4992 |
| *R3Avg* | 1239,6 | 2787,6 | 3869,1 | 4165,4 | 4699,4 |
| *R3Max* | 885,8 | 2098,2 | 2608,2 | 2819,2 | 3262,1 |
| *R3Min* | 1492,7 | 3287,2 | 4587,3 | 4956 | 5554,4 |
| *R2* | 1331,6 | 2965,8 | 4139,2 | 4449,5 | 4975,9 |
| *XDrawInt* | 1165,7 | 2731 | 3869 | 4290,2 | 5000,6 |
| *XDrawAvg* | 1234,4 | 2845,3 | 4065,8 | 4417,3 | 5105 |
| *XDrawMax* | 730 | 1688 | 2019,3 | 2040,2 | 2133,5 |
| *XDrawMin* | 1751 | 4776,1 | 7976,8 | 10495 | 12862,4 |
| *4WS* | 1061,1 | 2412,5 | 3124,4 | 3358,9 | 3832,4 |
| *8WS* | 1238,9 | 2776,4 | 3809,5 | 4049,6 | 4523,7 |

Table 2: Average number of visible points of viewshed algorithms at viewpoint height +1,6 m

| Algorithm\$r$ | 50 | 100 | 150 | 200 | 250 |
|---|---|---|---|---|---|
| *R3Avg* | 122,1 | 240,4 | 346,5 | 369,4 | 390,1 |
| *R3Max* | 454,4 | 881,5 | 1546,6 | 1642,7 | 1729,9 |
| *R3Min* | 0 | 0 | 0 | 0 | 0 |
| *R2* | 14,8 | 27 | 38,8 | 34,1 | 36,6 |
| *XDrawInt* | 274,5 | 642 | 1029 | 1183 | 1279 |
| *XDrawAvg* | 183,9 | 437,3 | 623,1 | 758,6 | 838,4 |
| *XDrawMax* | 610,2 | 1291,8 | 2135,7 | 2421,5 | 2858,7 |
| *XDrawMin* | 38,9 | 23,6 | 11,2 | 15,6 | 3,8 |
| *4WS* | 295,4 | 608,4 | 1092,6 | 1185,3 | 1253,5 |
| *8WS* | 127,1 | 258,5 | 427 | 508 | 582,4 |

Table 3: Average number of visible points that are invisible according to *R3Int*

The comparison of all viewshed algorithms is summarised in Tables 5 and 6 and in Figure 15. In Table 5, the comparison of average spent CPU time with the respect to algorithm *R3Int* is given. Similarily, Table 6 shows the average error ratio with the respect to all points in the viewshed. It can be observed that the error limits to zero with the growth of range $r$ for all algorithms. The most accurate algorithm is obvious *R3Int*. The least amount of CPU time needs algorithm *8WS* followed by algorithms *XDrawAvg* and *4WS*. If the results of other algorithms based on the method *R3* are ignored (because of the spent CPU time), the best solution is obtained by algorithm *R2* followed by

| Algorithm\r | 50 | 100 | 150 | 200 | 250 |
|---|---|---|---|---|---|
| *R3Avg* | 21,7 | 48,2 | 60,7 | 73 | 97,8 |
| *R3Max* | 0 | 0 | 0 | 0 | 0 |
| *R3Min* | 152,6 | 307,4 | 432,3 | 494,2 | 562,4 |
| *R2* | 6,1 | 13 | 23,2 | 22 | 20,3 |
| *XDrawInt* | 99,8 | 393,1 | 743,1 | 1011,3 | 1287,6 |
| *XDrawAvg* | 78 | 302,8 | 534,2 | 714,2 | 951,4 |
| *XDrawMax* | 0 | 0 | 0 | 0 | 0 |
| *XDrawMin* | 449,5 | 1820,2 | 3833,3 | 6048,9 | 7874,1 |
| *4WS* | 16,4 | 41 | 62,2 | 82,2 | 93,8 |
| *8WS* | 25,9 | 55,3 | 81,7 | 95,7 | 114,1 |

Table 4: Average number of invisible points that are visible according to *R3Int*

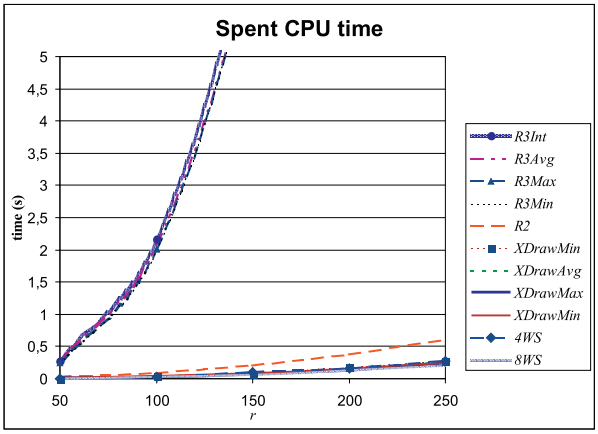



Figure 11: Average spent CPU times (s) of viewshed algorithms at viewpoint height +1,6 m

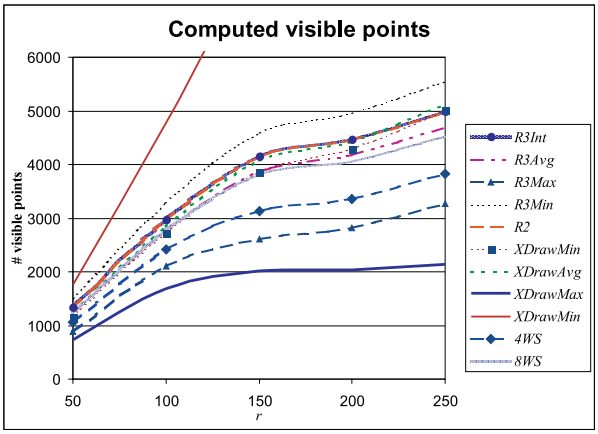



Figure 12: Average number of visible points of viewshed algorithms at viewpoint height +1,6 m

algorithms *8WS* and *4WS*.

Which algorithm among *8WS* and *R2* is the best or the most appropriate is difficult to claim. Namely, when selecting the algorithm, it is important to know which type of

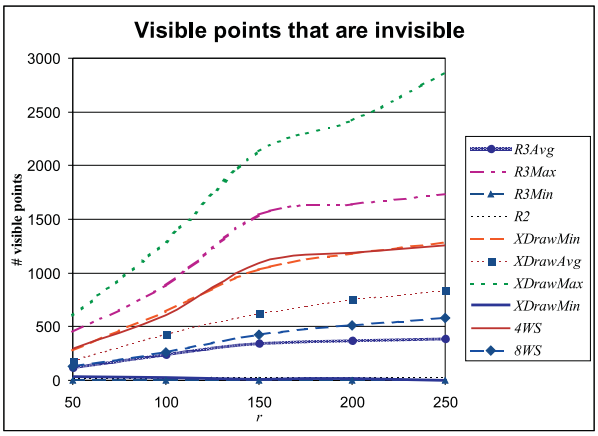



Figure 13: Average number of visible points that are invisible according to *R3Int*

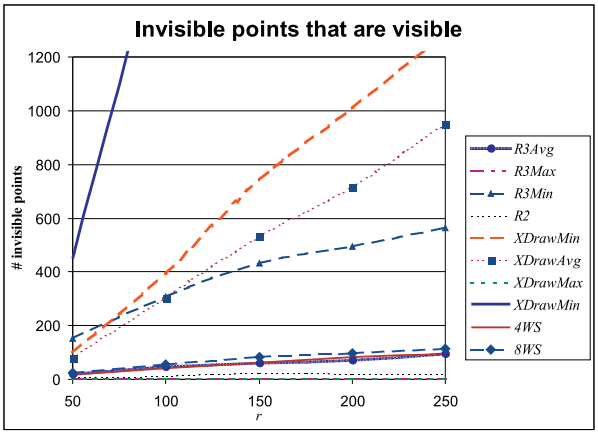



Figure 14: Average number of invisible points that are visible according to *R3Int*

error (e.g. visible points recognised as invisible) is allowed and which not. In some applications it is perhaps allowed to recognise invisible points as visible, and in some other applications the opposite.

| Algorithm | Time (%) |
|---|---|
| *R3Int* | 100 |
| *R3Avg* | 95,34 |
| *R3Min* | 92,11 |
| *R3Max* | 91,62 |
| *R2* | 3,65 |
| *XDrawInt* | 1,03 |
| *XDrawAvg* | 0,95 |
| *XDrawMin* | 1,52 |
| *XDrawMax* | 1,81 |
| *4WS* | 1,01 |
| *8WS* | 0,8 |

Table 5: Comparison (%) of average CPU times for viewshed algorithms

| Algorithm\r | 50 | 100 | 150 | 200 | 250 |
|---|---|---|---|---|---|
| *R3Avg* | 1,41 | 0,71 | 0,45 | 0,28 | 0,19 |
| *R3Max* | 4,45 | 2,18 | 1,71 | 1,02 | 0,69 |
| *R3Min* | 1,50 | 0,76 | 0,48 | 0,31 | 0,22 |
| *R2* | 0,20 | 0,10 | 0,07 | 0,03 | 0,02 |
| *XDrawInt* | 3,67 | 2,56 | 1,96 | 1,36 | 1,02 |
| *XDrawAvg* | 2,57 | 1,83 | 1,28 | 0,92 | 0,71 |
| *XDrawMax* | 5,98 | 3,20 | 2,36 | 1,51 | 1,14 |
| *XDrawMin* | 4,79 | 4,56 | 4,24 | 3,77 | 3,14 |
| *4WS* | 3,06 | 1,61 | 1,27 | 0,79 | 0,54 |
| *8WS* | 1,50 | 0,78 | 0,56 | 0,38 | 0,28 |

Table 6: Comparison (%) of average error ratio for viewshed algorithms

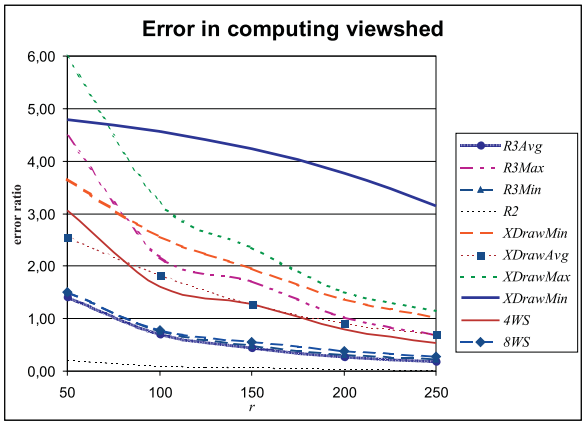



Figure 15: Comparison (%) of average error ratio for viewshed algorithms

# 5 Conclusions

New applications employing the visibility information arise daily. The basic visibility structure that is used in these applications is a viewshed. Few researchers have developed the methods for computing it, and the paper surveys them. Every method is presented shortly, implemented and experimentally compared using real terrain data and fractal generated terrains.

Because computing the viewshed is a demanding task, the selection of the most appropriate algorithm is important. The most accurate solution is obtained by the exact algorithm *R3Int*, but this algorithm is very slow. Luckily, all presented approximative algorithms work with relatively small error, and can be efficiently applied. However, it is important to determine which type of error in the approximative solutions is allowed and which not.

The challenge to improve those algorithms remains.

# References


[1] Daniel Cohen-Or, Amit Shaked. *Visibility and Dead-Zones in Digital Terrain Maps*. Computer Graphics Forum, 14(3), pp. 171–179, 1995.

[2] Leila De Floriani, Paola Magillo. *Intervisibility on Terrains*, Geographic Information Systems: Principles, Techniques, Management and Applications, Longley P.A., Goodchild M.F., Maguire D.J., Rhind D.W. (eds), John Wiley & Sons, pp. 543–556, 1999.

[3] Leila De Floriani, Enrico Puppo, Paola Magillo. *Applications of Computational Geometry to Geographic Information Systems*, Handbook of Computational Geometry, Sack J.R., Urrutia J. (eds), Elsevier Science, pp. 333–388, 1999.

[4] Wm Randolph Franklin, Clark K. Ray, Shashank Mehta. *Geometric Algorithms for Sitting of Air Defense Missile Batteries*, Research Project for Battelle, Columbus Division, Contract Number DAAL03-86-D-0001, Delivery Order Number 2756, 1994

[5] Wm Randolph Franklin, Clark K. Ray. *Higher isn't Necessarily Better: Visibility Algorithms and Experiments*, Advances in GIS Research: Proceedings of the 6th International Symposium on Spatial Data Handling, Waugh T.C., Healey R.G., (eds), Edinburgh, UK, pp. 751–770, 1994.

[6] Branko Kaučič, Borut Žalik. *An Overview of Visibility Problem Algorithms in 1,5D*, Proceedings of the 10th International Conference in Central Europe on Computer Graphics, Visualization and Computer Vision'2002, Skala V. (ed) pp. 241–247, 2002.

[7] Heinz-Otto Peitgen. *The Science of Fractal Images*, Springer-Verlag, 1988.

[8] Tomaž Trobec, Borut Žalik, Nikola Guid. *Two Algorithms for Visibility Determination of Raster Relief Models*, Spring Conference on Computer Graphics SCCG'98, Kalos L.S. (ed), Budmerice, Slovak Republic, pp. 247–256, 1998.

[9] Jerry Van Aken, Mark Novak. *Curve-drawing algorithms for raster displays*, ACM Transactions on Graphics, 4(2), pp. 147–169, 1985

[10] http://www.gov.si/gu/ [20.02.2002]



Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
Spring Conference on Computer Graphics, Budmerice, Slovakia
Copyright © ACM 2002 1-58113-608-0/02/009...$5.00